软件过程与管理课程作业（基于《空箱》项目）

1. 作业说明

本文基于本人参与开发的《空箱》Unity 项目，结合课堂讲授的 ISO/IEC 12207、ISO/IEC 15504、ISO/IEC 33001 内容，给出该项目的软件过程定义，并说明过程定义来源与剪裁思路。

---

2. 项目背景与对象

- **项目名称**：`空箱`
- **项目背景**：参加 GameJam 活动 **Game 4（For） Good Jam** 的作品
- **技术栈**：Unity 2022.3、C#、QFramework、UniTask、Addressables、YarnSpinner
- **项目类型**：课程/练习性质的 2D 游戏项目（以关卡推进与交互流程为核心）
- **项目特征**：
  - 以玩法原型和迭代速度优先；
  - 使用 JSON 进行关卡数据描述；
  - 采用 `GameFlowController` 组织“关卡加载-对话-等待-信号”流程；
  - 架构上使用 QFramework 的 Model/System/Command 思想实现业务解耦。

---

3. 软件过程定义（面向本项目）

结合项目实际，我将软件过程分为 6 个主过程与 3 个支撑过程。

3.1 主过程（Primary）

P1 立项与范围定义过程

- **目标**：明确本轮迭代要实现的玩法目标与验收条件。
- **输入**：课程要求、创意点子、现有代码状态。
- **活动**：
  - 定义本轮目标（如新增关卡流程能力、对话推进能力）；
  - 拆分为最小可交付功能（MVP）；
  - 形成任务清单（功能、数据、UI、调试项）。
- **输出**：迭代任务列表、验收标准。

P2 需求与规则建模过程

- **目标**：将“玩法想法”转化为可编码规则。
- **输入**：玩法描述、上一版反馈。
- **活动**：
  - 提炼核心实体（Player/Box/Obstacle/Pill）；

    - 定义状态与约束（关卡尺寸、终点、移动规则、完成条件）；
    - 设计关卡 JSON 结构（LevelData、EntityData、EndPoints）。
- **输出**：规则定义、数据结构定义、示例关卡数据。

P3 架构与详细设计过程

- **目标**：保证扩展性与可维护性。
- **输入**：需求模型、已有框架能力。
- **活动**：
    - 基于 QFramework 定义架构入口（`GameArchitecture`）；
    - 划分职责：Model（状态）、System（系统能力）、Command（动作封装）、Mono（场景交互）；
    - 设计流程编排器（`GameFlowController`）与步骤模型（LoadLevel/Dialogue/Wait/Signal）；
    - 定义事件驱动更新机制（实体增删改、视图重建）。
- **输出**：模块边界、类职责、关键接口。

P4 实现与自测过程

- **目标**：快速实现并持续验证核心功能可用。
- **输入**：设计结果、任务清单。
- **活动**：
    - 编写或修改脚本（核心逻辑、流程控制、关卡加载）；
    - 在 Unity Editor 中进行场景联调；
    - 通过运行日志与断点完成问题定位；
    - 对关键流程进行回归验证（重开关卡、通关判断、对话推进）。
- **输出**：可运行功能版本、缺陷清单与修复记录。

P5 集成与验收过程

- **目标**：确保多个子功能在一个可玩的闭环中稳定工作。
- **输入**：各模块实现结果。
- **活动**：
    - 集成关卡数据、实体系统、UI 展示、流程控制；
    - 验证主路径：进入场景 -> 加载关卡 -> 交互推进 -> 达成条件；
    - 验证异常路径：关卡文件缺失、输入未配置、信号未触发等。
- **输出**：阶段性可交付版本、验收结果。

P6 迭代复盘与演进过程

- **目标**：将问题沉淀为后续迭代改进项。
- **输入**：运行结果、测试观察、成员反馈。
- **活动**：
    - 记录高频问题（耦合点、重复逻辑、配置复杂度）；

    - 形成下一轮优化计划（性能、可读性、可配置性）；
    - 同步更新文档与数据模板。
- **输出**：复盘结论、下轮迭代计划。

3.2 支撑过程（Supporting）

S1 配置与版本管理

- Git 管理脚本与配置变更；
- 约定目录结构（`_ProjectAssets/Scripts`、`_ProjectAssets/Data`）；
- 通过提交记录跟踪问题来源。

S2 质量保证与验证

- 以场景联调和回归清单为主；
- 对关键逻辑设置日志与错误提示（如关卡文件路径错误、反序列化失败）；
- 通过“主流程+异常流程”双轨检查提升稳定性。

S3 文档与知识沉淀

- 维护关卡数据格式约定；
- 记录流程步骤配置说明；
- 形成可复用的开发约定（命名、目录、模块职责）。

---

4. 软件过程定义的来源说明

本项目的软件过程定义主要来自以下三类来源：

4.1 来源一：ISO/IEC 12207（过程框架来源）

- 采用其“生命周期过程分类”思想，将项目活动分为主过程与支撑过程；
- 参考其“需求、设计、实现、集成、验证、维护”等活动链路；
- 结合课程内容，不照搬标准条目，而是将标准思想映射到 Unity 小团队项目实践。

4.2 来源二：ISO/IEC 15504（过程能力评估来源）

- 采用“过程能力分级”思想评价当前实践成熟度；
- 本项目侧重达到“过程已执行、基本可管理”的水平（有明确活动、输入输出、责任分工）；
- 将“可重复、可检查”作为过程定义约束（例如每次迭代都执行主路径回归）。

4.3 来源三：ISO/IEC 33001（过程改进与评估体系来源）

- 采用其“以评估驱动改进”的思路；
- 将复盘结果作为下一轮过程改进输入，形成闭环；
- 将“过程定义不是一次性文档，而是持续演进资产”作为执行原则。

---

5. 软件过程定义的层次性

结合课堂“过程定义层次化”内容，本项目按三层定义：

- **L1 标准层（Why）**：遵循 12207/15504/33001 的通用理念，明确为什么要有过程；
- **L2 项目层（What）**：定义本项目需要执行哪些过程（P1-P6、S1-S3）；
- **L3 实施层（How）**：落实到具体脚本、目录、配置、检查清单与操作步骤（如关卡 JSON 校验、流程步骤联调、异常路径验证）。

---

6. 过程剪裁思路（Tailoring）

考虑到《空箱》属于 GameJam 场景下的课程/原型型项目，团队规模小、周期紧，本项目采用“保主干、轻流程、重反馈”的剪裁策略：

6.1 保留的过程

- 需求建模、架构设计、实现、自测、集成、复盘全部保留；
- 配置管理与质量验证保留；
- 原因：这些过程直接影响可运行性与迭代效率，不能省略。

6.2 弱化或合并的过程

- 将正式评审、复杂度量统计、大规模文档审批进行简化；
- 将详细需求评审与设计评审合并为一次迭代评审；
- 原因：项目规模较小，过重流程会抬高沟通成本，影响迭代速度。

6.3 暂不纳入的过程

- 采购管理、供应商管理、正式发布运维流程（按企业级标准）暂不纳入；
- 原因：课程项目无外部采购链条、无商用上线运维场景。

6.4 剪裁后的收益与风险

- **收益**：研发节奏更快、沟通链条更短、实验空间更大；
- **风险**：需求追踪与变更留痕可能不足、测试覆盖可能偏经验化；
- **控制措施**：用最小文档模板 + 回归清单 + 关键日志策略来抵消风险。

---

7. 结论

本次软件过程定义基于课堂标准（12207/15504/33001）与项目实际（Unity 原型开发）进行了层次化落地和针对性剪裁。
最终形成了“可执行、可检查、可迭代改进”的轻量过程框架，既满足课程作业要求，也能真实指导该项目后续开发。